

## Qué son

Las **listas** en Python son **colecciones ordenadas y mutables** que permiten almacenar múltiples elementos (números, textos, booleanos, etc.) en una sola variable.

👉 Se definen con corchetes `[]`.

```
frutas = ["manzana", "pera", "naranja"]
```

Son **esenciales** en proyectos modulares porque permiten modificar, ordenar y buscar elementos de forma eficiente durante la ejecución del programa

### ◆ Tipos de listas

| Tipo de lista               | Descripción                                                     | Ejemplo                                             |
|-----------------------------|-----------------------------------------------------------------|-----------------------------------------------------|
| <b>Simple</b> s             | Contienen elementos del mismo tipo (homogéneos).                | <code>[1, 2, 3, 4, 5]</code>                        |
| <b>De diferentes tipos</b>  | Mezclan varios tipos de datos (enteros, cadenas, booleanos...). | <code>[23, "texto", True, 3.14]</code>              |
| <b>Anidadas o complejas</b> | Contienen otras listas o estructuras dentro de sí.              | <code>usuarios = [{"Juan", 30}, {"Ana", 25}]</code> |

*Las listas simples son más rápidas; las anidadas organizan mejor datos jerárquicos, y las mixtas son más flexibles.*

## Propiedades clave

- **Ordenadas:** mantienen el orden de inserción de los elementos.
- **Mutables:** pueden modificarse (añadir, eliminar o cambiar valores).
- **Admiten duplicados:** puedes tener varios elementos iguales.
- **Indexadas:** se accede a sus elementos por su posición.

## Acceso y modificación de elementos

| Operación            | Descripción                                                 | Ejemplo                                      |
|----------------------|-------------------------------------------------------------|----------------------------------------------|
| Acceso por índice    | Accede al elemento en una posición concreta (empieza en 0). | <code>mi_lista[1]</code>                     |
| Modificación directa | Permite cambiar un elemento.                                | <code>mi_lista[2] = "nuevo"</code>           |
| Slicing (sublistas)  | Extrae o modifica partes de la lista.                       | <code>mi_lista[1:3],<br/>mi_lista[:2]</code> |
| Índices negativos    | Permiten acceder desde el final.                            | <code>mi_lista[-1]</code>                    |

 Ejemplo:

```
colores = ["rojo", "verde", "azul"]
colores[1] = "amarillo" # cambia 'verde' por 'amarillo'
print(colores)
# ['rojo', 'amarillo', 'azul']
```

## Métodos comunes de manipulación

| Método                    | Descripción                                                                                                                             | Ejemplo                                                                         | Ejemplo de resultado         |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|------------------------------|
| <code>append(x)</code>    | Agrega <b>un</b> elemento al final de la lista. <b>Si x es una lista, se añade como sublista</b> → lista anidada                        | <code>mi_lista.append(8)</code><br><code>mi_lista.append([1,2,3])</code>        | <code>[8, [1,2,3]]</code>    |
| <code>extend(x)</code>    | Agrega <b>varios</b> elementos al final de la lista. <b>Si x es una lista, agrega cada elemento individualmente</b> → lista plana/mixta | <code>mi_lista.extend([8,9,10])</code><br><code>mi_lista.extend([1,2,3])</code> | <code>[8,9,10,1,2,3]</code>  |
| <code>insert(i, x)</code> | Inserta un elemento en una posición concreta.                                                                                           | <code>mi_lista.insert(2, "dato")</code>                                         | <code>[8,9,"dato",10]</code> |
| <code>remove(x)</code>    | Elimina el primer valor igual a x.                                                                                                      | <code>mi_lista.remove("dato")</code>                                            | <code>[8,9,10]</code>        |

|           |                                                                                       |                                          |                                                      |
|-----------|---------------------------------------------------------------------------------------|------------------------------------------|------------------------------------------------------|
| pop(i)    | Elimina el elemento en la posición i y lo devuelve. Sin argumento, elimina el último. | <code>eliminado = mi_lista.pop(1)</code> | <code>eliminado=9</code> y <code>lista=[8,10]</code> |
| clear()   | Vacía toda la lista.                                                                  | <code>mi_lista.clear()</code>            | <code>[]</code>                                      |
| index(x)  | Devuelve la posición de un valor.                                                     | <code>mi_lista.index(3)</code>           | <code>2</code>                                       |
| count(x)  | Cuenta cuántas veces aparece un valor.                                                | <code>mi_lista.count("rojo")</code>      | <code>1</code>                                       |
| sort()    | Ordena la lista (alfabéticamente o numéricamente).                                    | <code>mi_lista.sort()</code>             | <code>[1,2,3,8,10]</code>                            |
| reverse() | Invierte el orden de los elementos.                                                   | <code>mi_lista.reverse()</code>          | <code>[10,8,3,2,1]</code>                            |
| len()     | Devuelve el número de elementos.                                                      | <code>len(mi_lista)</code>               | <code>5</code>                                       |
| in        | Comprueba si un valor existe.                                                         | <code>"rojo" in colores</code>           | <code>True</code>                                    |

Estos métodos permiten modificar listas sin crear nuevas estructuras, ahorrando memoria y tiempo

## Ordenación

| Técnica                                                | Descripción                                                                                                    | Ejemplo                                                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sort()</code>                                    | Ordena la lista directamente.                                                                                  | <code>numeros.sort();</code> ✓ <b>Modifica la lista original</b><br>✓ <b>No devuelve nada</b> (devuelve <code>None</code> )<br>✓ Ordena la lista <i>en el sitio</i> (in-place)<br>✓ Para ordenar en sentido inverso se puede usar <code>reverse=True</code> ;<br><code>numeros.sort(reverse=True)</code>     |
| <code>sorted(lista)</code>                             | Crea una nueva lista ordenada, no modifica la original.                                                        | <code>sorted(numeros)</code> ✓ <b>NO modifica la lista original</b><br>✓ <b>Devuelve una nueva lista ordenada</b><br>✓ Sirve para mantener intacta la lista original<br>✓ Para ordenar en sentido inverso se puede usar <code>reverse=True</code> ;<br><code>nueva_lista = sorted(lista, key=funcion)</code> |
| Ordenación con <code>key</code> - <code>reverse</code> | <code>key</code> → indica <b>por qué criterio</b> se ordena, ojo! <code>key</code> siempre espera una función. | <code>palabras = ["sol", "estrella", "luna", "planeta"]</code><br><code>palabras.sort(key=len)</code><br><code>print(palabras)</code><br><code>['sol', 'luna', 'planeta', 'estrella']</code><br>Otro ejemplo:<br><code>nombres = ["ana", "Luis", "carlos", "Bea"]</code>                                     |

|  |                                                                             |                                                                                                                                                                                                                                                                                                                      |
|--|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <code>reverse=True</code><br>→ indica el<br><b>sentido</b><br>(descendente) | <code>nombres.sort(key=str.lower) #ignora mayúsculas</code><br><code>print(nombres)</code><br><code>['ana', 'Bea', 'carlos', 'Luis']</code><br>También se puede combinar, key y reverse:<br><code>palabras.sort(key=len, reverse=True)</code><br><code>nueva_lista = sorted(lista, key=funcion, reverse=True)</code> |
|--|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## BÚSQUEDA EN LISTAS SIMPLES, MIXTAS Y ANIDADAS

En Python, para buscar elementos dentro de una lista usamos normalmente el operador `in`, que realiza una búsqueda secuencial: recorre internamente la lista elemento por elemento hasta encontrar una coincidencia o confirmar que no está.

Sin embargo, su comportamiento cambia según el tipo de lista:

### ♦ 1. Búsqueda en listas simples

En una lista “normal” (no anidada), `in` busca directamente entre los elementos:

```
animales = ["gato", "perro", "loro", "pez"]

if "loro" in animales:

    print("loro, Está en la lista")
```

📌 Python recorre internamente todos los elementos:

¿`"loro" == "loro"`? ✓ → se detiene

Este comportamiento es válido también para listas mixtas:

```
mixta = [10, "hola", True]

if "hola" in mixta:

    print("Encontrado")
```

### ♦ 2. Búsqueda en listas anidadas (primer nivel)

En una lista anidada:

```
anidada = [[1, 2], ["a", "b"], [True, False]]
```

# Aquí 1 NO está en el primer nivel, solo hay sublistas

```
if 1 in anidada:

    print("1 está en la lista anidada (primer nivel).")

else:

    print("1 NO está en la lista anidada (primer nivel).")

# Aquí buscamos la SUBLISTA completa [1, 2]

if [1, 2] in anidada:

    print("[1, 2] SÍ está como sublista en la lista anidada.")

else:

    print("[1, 2] NO está como sublista en la lista anidada.")
```

Salida:

1 NO está en la lista anidada (primer nivel).

[1, 2] SÍ está como sublista en la lista anidada.

### ♦ 3. Buscar elementos dentro de sublistas

Si queremos buscar dentro de cada sublista, necesitamos recorrerlas.  
Aquí es donde entra en juego el for.

#### ✓ Opción A — Un solo **for** + **in** dentro

```
anidada = [[1, 2], ["a", "b"], [True, False]]

buscado = "b"

for sublista in anidada:

    if buscado in sublista:        # ahora sí se busca dentro de cada sublista

        print("Encontrado dentro de una sublista")

        break
```

Python revisa:

`[1,2]` → no está

`["a","b"]` → en esta sublista sí está "b"

---

✓ Opción B — Doble **for** (más detallado y otro modo de hacerlo)

```
encontrado = False
```

```
for sublista in anidada:
```

```
    for elemento in sublista:
```

```
        if elemento == buscado:
```

```
            encontrado = True
```

```
            break
```

```
if encontrado:
```

```
    break
```

---

#### ◆ 4. Buscar una sublista completa

Si buscáramos una sublista completa, sí podemos usar directamente **if ... in**:

```
if [1,2] in anidada:
```

```
    print("Sublista encontrada")
```

No hace falta for.

**Búsqueda binaria:**

La **búsqueda binaria** sirve para **encontrar rápidamente un elemento en una lista ordenada**.

En lugar de revisar los elementos uno por uno (como en la búsqueda secuencial), la búsqueda binaria **divide la lista a la mitad en cada paso**, reduciendo el número de comparaciones.

Funciona **solo si la lista ya está ordenada** (por ejemplo, con `.sort()` o `sorted()`).

## Cómo funciona:

Tienes una lista **ordenada**. Ejemplo:

```
numeros = [1, 3, 5, 7, 9, 11, 13]
```

1. Buscas un valor, por ejemplo 7.
  - Empieza en el **centro** de la lista.
  - Si el número buscado es menor, descarta la mitad derecha.
  - Si es mayor, descarta la mitad izquierda.
  - Repite el proceso hasta encontrarlo.
2. Repite el proceso con la mitad correspondiente hasta encontrar el valor o agotar la lista.

## Resumen

|                     |                                            |                                   |
|---------------------|--------------------------------------------|-----------------------------------|
| Búsqueda secuencial | Recorre la lista elemento a elemento.      | <code>if "Ana" in lista:</code>   |
| Búsqueda binaria    | Requiere lista ordenada, busca más rápido. | (con módulo <code>bisect</code> ) |

También puedes usar `min()`, `max()` o `sum()` para obtener valores específicos

```
**palabras = ["sol", "estrella", "luna", "planeta"]
```

Aquí `palabras` es la lista que queremos ordenar. Cada elemento de `palabras` es una **cadena de texto**, como `"sol"` o `"estrella"`. En:

```
palabras.sort(key=lambda palabra: len(palabra))
```

El nombre `palabra` **solo existe dentro del `lambda`**. Es como una **variable temporal** que representa cada elemento de la lista mientras `sort` recorre la lista.

## IMPORTANTE sobre ordenar listas

- **sort()** solo funciona con listas cuyos elementos se pueden comparar entre sí, por ejemplo:

- números con números
- cadenas con cadenas

✗ **No funciona con listas mixtas** (porque no puede comparar "hola" con 10)

✗ **No funciona con listas anidadas** con **sort()** (porque no sabe cómo comparar sublistas entre sí), lo único que podemos hacer en este caso es **ordenar las sublistas si no son mixtas**. Cuando Python ordena una lista con **.sort()** o **sorted()**, compara los elementos de la lista **entre sí** y los coloca en el orden adecuado según su valor. Si los elementos son números, los ordena de menor a mayor; si son cadenas, las ordena alfabéticamente. Cuando se trata de una lista anidada (listas dentro de otra lista), Python **no ordena el contenido interno de cada sublista**, sino que compara cada sublista como un bloque y decide su posición comparando primero su *primer elemento*; si estos son iguales, compara el segundo, y así sucesivamente. Por eso, para que una lista anidada se pueda ordenar, **todas las sublistas deben contener elementos del mismo tipo**, ya que Python necesita poder compararlas entre sí. En resumen, Python ordena moviendo los elementos completos de las sublistas, manteniendo intacto su contenido interno.

| Tipo de lista         | ¿Se puede ordenar con <b>.sort()</b> ? | Explicación                                                                                      |
|-----------------------|----------------------------------------|--------------------------------------------------------------------------------------------------|
| Simple (solo números) | ✓ Sí                                   | Todos son comparables                                                                            |
| Solo strings          | ✓ Sí                                   | Se ordenan alfabéticamente                                                                       |
| Anidadas              | ✓ Sí, si las sublistas son comparables | Python compara listas entre sí. Todas las sublistas deben ser del mismo tipo, números o strings. |
| Mixtas                | ✗ No                                   | No se puede comparar "hola" con 10                                                               |

## Iteración y filtrado

Las listas pueden recorrerse con bucles o filtrarse con comprensión de listas.

### ♦ Iterar (recorrer) con **for**

```
for elemento in mi_lista:  
    print(elemento)
```

El bucle **for** recorre (**itera**) la lista **mi\_lista** elemento por elemento. En cada vuelta del bucle, la variable **elemento** **toma el valor** del siguiente elemento de la lista. Ejemplo:

```
mi_lista = ["rojo", "verde", "azul"]  
for elemento in mi_lista:  
    print(elemento)  
rojo  
verde  
azul
```

### ♦ Filtrar con comprensión de listas

```
pares = [n for n in range(10) if n % 2 == 0] #[nueva_variable for variable in secuencia if condición]  
print(pares) # [0, 2, 4, 6, 8]
```

## Qué hace

① `range(10)`

→ genera los números del 0 al 9 (no incluye el 10).

 `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`

② `for n in range(10)`

→ recorre cada número del rango.

③ `if n % 2 == 0`

→ solo selecciona los que son **pares** (% calcula el resto de la división entre 2).

- Si el resto es 0 → el número es par.

④ `[n for n in range(10) if n % 2 == 0]`

→ crea una lista con todos los `n` que cumplen la condición.

Resultado final

```
[0, 2, 4, 6, 8]
```

La primera **n** en esta expresión:

```
pares = [n for n in range(10) if n % 2 == 0]
```

es lo que se va a guardar en la lista.

## ◆ Funciones útiles

### Funciones útiles

| Función         | Descripción                                                                                                                                                                                             | Ejemplo                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Devuelve     |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| <b>map()</b>    | Aplica una función a todos los elementos. Ojo! map requiere como argumento obligatoriamente una función.                                                                                                | <pre>def doble(x):<br/>    return x * 2<br/><br/>numeros = [1, 2, 3, 4]<br/>resultado = list(map(doble, numeros))<br/>#map() aplica la función doble() a cada elemento de la lista numeros.map() no devuelve directamente una lista, sino un objeto iterable (una especie de "receta" que genera los valores uno a uno). Por eso se usa list() para convertir ese resultado en una lista real que se pueda imprimir o usar.</pre>                         | [2, 4, 6, 8] |
| <b>filter()</b> | Recorre TODOS los elementos de la lista, uno por uno, y solo deja pasar aquellos que cumplen la condición de la función que le pasas. Ojo! filter requiere como argumento obligatoriamente una función. | <pre>def mayor_que_cinco(x):<br/>    return x &gt; 5<br/>#Esa función recibe un número (x) y devuelve True o False según si el número es mayor que 5.<br/><br/>numeros = [2, 7, 4, 9, 3, 8]<br/>resultado = list(filter(mayor_que_cinco, numeros))<br/>#filter() siempre trabaja con valores booleanos, en nuestro ejemplo aplica la función mayor_que_cinco a cada elemento de la lista numeros.<br/>Solo mantiene los valores que devuelven True.</pre> | [7, 9, 8]    |

Estas funciones son ideales para procesar grandes volúmenes de datos sin usar bucles manuales.

## Resumen general

| Propiedad / Operación | Ejemplo de uso                              |
|-----------------------|---------------------------------------------|
| Crear lista           | <code>lista = [1, 2, 3]</code>              |
| Añadir elemento       | <code>lista.append(4)</code>                |
| Insertar elemento     | <code>lista.insert(1, "dato")</code>        |
| Eliminar por valor    | <code>lista.remove("dato")</code>           |
| Eliminar por índice   | <code>lista.pop(2)</code>                   |
| Acceder por índice    | <code>lista[0]</code>                       |
| Sublista (slicing)    | <code>lista[1:3]</code>                     |
| Buscar elemento       | <code>"rojo" in lista</code>                |
| Contar elementos      | <code>len(lista)</code>                     |
| Ordenar               | <code>lista.sort()</code>                   |
| Invertir              | <code>lista.reverse()</code>                |
| Filtrar               | <code>[x for x in lista if x &gt; 5]</code> |

## Conclusión

Las **listas en Python** son una herramienta **versátil, flexible y potente** para almacenar y manipular datos.

Gracias a su **mutabilidad**, pueden crecer, cambiar y adaptarse durante la ejecución del programa.

Dominar sus **métodos** (como `append`, `insert`, `remove`, `sort`, `index`, `count`, etc.) es clave para desarrollar código **eficiente, limpio y escalable**.

## EJERCICIOS CON LISTAS



## EJERCICIO 1 — Crear y mostrar una lista de cada tipo

**Objetivo:** Crear una lista simple, otra mixta y otra anidada

---



## EJERCICIO 2 — Recorrer listas

**Objetivo:** Imprimir cada elemento de una lista simple, una lista mixta y una anidada con un `for`.

```
frutas = ["manzana", "pera", "plátano", "naranja"]
for fruta in frutas:
    print("Fruta:", fruta)
```

## EJERCICIO 3 — Añadir y eliminar elementos

Escribe un programa que añada y elimine elementos de listas. Debe trabajar con los **tres tipos de listas**:

1. **Lista simple** → contiene solo números.
  - Se añadirá un elemento con `append()`
  - Se eliminará un elemento existente con `remove()`
2. **Lista mixta** → contiene valores de distintos tipos (números, cadenas, booleanos...).
  - Se añadirá un nuevo valor de cualquier tipo
  - Se eliminará un valor específico
3. **Lista anidada** → contiene otras listas dentro.
  - Se añadirá una **sublista completa**
  - Se eliminará otra sublista usando `remove()`

El programa debe mostrar **el estado de cada lista antes y después** de aplicar las operaciones, para que el alumno visualice claramente cómo cambian.

**Objetivo:** Usar `append()` y `remove()`.

```
colores = ["rojo", "verde", "azul"]
colores.append("amarillo")      # Añadir un color
print("Después de añadir:", colores)

colores.remove("verde")         # Eliminar un color
print("Después de eliminar:", colores)
```

## EJERCICIO 4 — Ordenar listas

**Objetivo:** Escribe un programa en Python que trabaje con diferentes tipos de listas (simples, mixtas y anidadas) para demostrar la diferencia entre los métodos `sort()`, `sorted()` y `reverse()`. El programa debe mostrar cómo `sort()` ordena modificando la lista original, mientras que `sorted()` devuelve una nueva lista sin alterar la inicial, incluyendo su uso con `reverse=True`. También debe intentar ordenar listas que no se pueden ordenar, como listas mixtas o anidadas con tipos incompatibles, usando `try/except` para capturar los errores. Finalmente, debe ordenar correctamente una lista anidada compuesta solo por sublistas numéricas y mostrar los resultados de cada operación para comparar el comportamiento de `sort()`, `sorted()` y `reverse()`.

Nota: `lista_numeros = numeros[:]` # crea una copia completa e independiente de la lista `numeros`.

## EJERCICIO 5 – Buscar un elemento

Escribe un programa en Python que permita al usuario buscar elementos en diferentes tipos de listas: una lista simple de elementos homogéneos, una lista mixta con distintos tipos de datos y una lista anidada (listas dentro de listas). Para la lista simple y la lista mixta, el programa debe usar `if ... in ...` para comprobar si el elemento ingresado por el usuario está presente. Para la lista anidada, debe recorrer cada sublista y cada elemento con un bucle `for` anidado para determinar si el elemento ingresado se encuentra dentro de alguna sublista, y mostrar un mensaje indicando si se encontró o no. El programa debe imprimir claramente los resultados de cada búsqueda.

El operador `"in"` recorre internamente toda la lista elemento por elemento o hasta encontrar una coincidencia (o confirmar que no está).

## EJERCICIO 6

Usando las funciones `map()` y `filter()`, escribe un programa en Python que:

1. A partir de una lista de números, genere **otra lista con el doble** de cada elemento.
2. Dada una segunda lista, obtenga **una nueva lista que contenga solo los números mayores que 5**.

Define las funciones necesarias y muestra las listas originales y las resultantes.

```
def doble(x): #función doble(x) que simplemente devuelve x*2

    return x*2
```

## EJERCICIO 7

Escribe un programa en Python que realice lo siguiente:

### 1 . Lista simple:

Solicita al usuario que ingrese 5 números enteros y crea una lista que los contenga. Luego muéstrala por pantalla.

### 2 . Lista mixta:

Pide al usuario su nombre, su edad y su ciudad, y crea una lista que combine estos datos (distintos tipos). Muestra la lista resultante.

### 3 . Lista anidada:

Solicita al usuario que ingrese el nombre y la nota de tres estudiantes.  
Con esta información, crea una lista anidada donde cada elemento sea a su vez una lista con la forma:  
["nombre\_del\_estudiante", nota]  
Muestra la lista anidada completa.

Añade código a este programa para pedir los datos a través de un bucle .

## EJERCICIO 8.

## 8.A Gestión de árboles: decidir el abono según altura y edad. Listas simples paralelas.

### Objetivo:

Pedir datos de varios árboles (altura y edad), guardarlos en listas, y decidir **qué abono necesitan** según sus características.

#### ♦ Criterios de ejemplo:

- Si **mide más de 10 metros** → abono tipo **A**
- Si **tiene más de 20 años** → abono tipo **B**
- Si cumple las dos → abono **A + B**
- Si no cumple nada → **sin abono especial**

## 8.B Gestión de árboles: Realizar el mismo ejercicio pero creando listas anidadas.

## EJERCICIO 9

Escribe un programa en Python que gestione un registro de personas usando una lista anidada, donde cada sublista contiene el **nombre, año de nacimiento y número de vacunas** de cada persona. El programa debe:

1. Filtrar y mostrar los **menores de 18** años que tengan menos de 3 vacunas, sin añadir aún la edad a las sublistas.
2. Añadir la edad de cada persona como un nuevo campo en la sublista, calculada a partir del año actual.
3. Mostrar un resumen completo de todas las personas, ordenado de menor a mayor edad, indicando el nombre, edad y número de vacunas.

```
personas = [  
    ["Ana", 2015, 2],  
    ["Luis", 2008, 4],  
    ["Marta", 2012, 1],  
    ["Carlos", 2005, 3],
```

```
["Lucía", 2017, 2],  
["Diego", 2010, 3],  
["Elena", 2014, 1],  
["Sofía", 2011, 2],  
["Javier", 2007, 4],  
["Clara", 2016, 1]  
]
```

## EJERCICIO 10 – Estadísticas básicas con listas

**Objetivo:** Practicar recorridos, acumuladores y funciones útiles (`sum`, `min`, `max`, `len`).

Dada una lista de números enteros:

```
numeros = [4, 7, 2, 9, 5, 10, 3]
```

El programa debe:

1. Mostrar la lista original.
2. Calcular y mostrar:
  - La suma total de los números.
  - El valor mínimo.
  - El valor máximo.
  - La media (promedio).
3. Mostrar cuántos números son mayores que la media y qué número son

## EJERCICIO 11 – Transformar y filtrar listas de textos

**Objetivo:**

Trabajar con listas, bucles, condiciones y comprensión de listas.

Dada la lista:

```
palabras = ["Python", "sol", "PROGRAMACIÓN", "lista", "DATOS", "ai"]
```

El programa debe:

1. Crear una nueva lista con todas las palabras en **minúsculas**.
2. A partir de esa lista, crear otra con **sólo las palabras que tengan más de 4 letras**.
3. Mostrar:
  - Lista original.
  - Lista en minúsculas.
  - Lista filtrada final.

## EJERCICIO 12 – Gestión de productos (listas anidadas)

### Objetivo:

Reforzar listas anidadas, condiciones, recorridos y ordenación.

### Enunciado

Dada la lista de productos:

```
productos = [  
    ["Pan", 1.20, 30],  
    ["Leche", 0.95, 10],  
    ["Huevos", 2.50, 0],  
    ["Arroz", 1.80, 25],  
    ["Aceite", 6.50, 5]  
]
```

El programa debe:

1. Mostrar todos los productos con su nombre, precio y stock.
2. Mostrar solo los productos **sin stock**.
3. Crear una nueva lista con los productos cuyo **precio sea mayor de 2 €**.
4. Ordenar los productos **por precio de menor a mayor**